

MBSD Miss Ramish. CS-301 CLO 2 & CLO 3

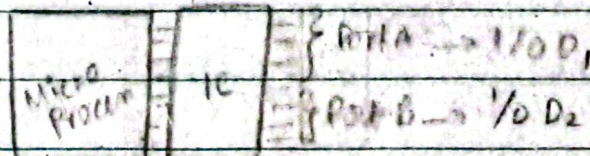
(Week 1) OLD — MBSD — DSD (Digital Sys. Design)

- COO: Hardwired MBSD: Hardware + Software
- ISA is req. for assembly lang.
- Internet of Things. (network actually not always internet)

Jan 8, 2026.

→ Microcontroller consist of CPU.

- With CPU, things have to be connected via ICs. But on micro-controller, everything is on the chip hence, comparatively fast.



- micro-processor has serial ports in place of ICs to ease the work.

Transferring Data
(Interfacing Circuitry)
needed, cannot be connected directly.

(Week 12) Jan 14, 2026

x = port name
n = bit number.

- Other than AVR, everything is peripheral.
- Digital Ports = I/O ports.
- All micro-processors, ICs are sequential.
- XTAL = crystal. Quad (not included)
- 35 I/O lines: Ports (A, B, C, D: PA, PB, PC, PD) (8-lines) each Port E (3 lines) each

Hence, we have 8bit format. we can have 16 n-bit micro-controllers as well.

- Integrated: Not a peripheral
- Ports name: x (A, B, C, D, E) → any port we can use
- Data Direction Register: DDRx
- DDA: 1 flipflop.
- DDxn: 0: pin xn = input line
1: pin xn = output line.

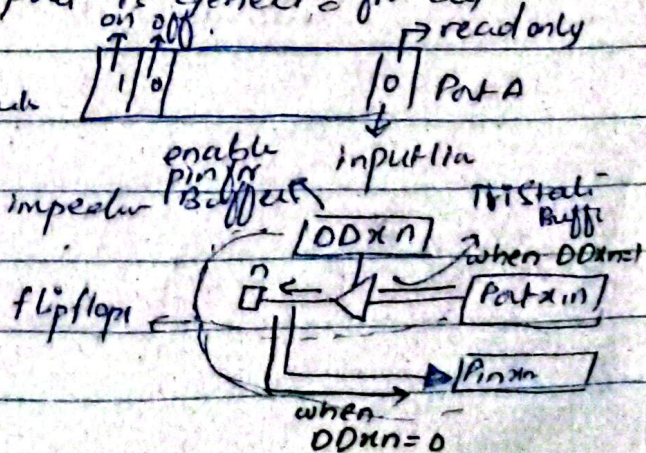
→ Register description for I/O ports is generic for all ports.

- If Input declared: use PINA.

- If Output declared: use PORTA.

→ TriState Buffer acts as high impedance circuitry when off. Buffer is complement of NOT gate.

- It has enabled pin



MBSD-MR-1

★
 100%!
 Come when
 mBIO with MS d
 Pan's

Enable pin: DDxn.

30 Buffer

So at DDxn → 0 - Buffer: High Impedance
 → 1 - Buffer on, transfer what
 is in pin

→ PINA is read-only register

Jan 16, 2026

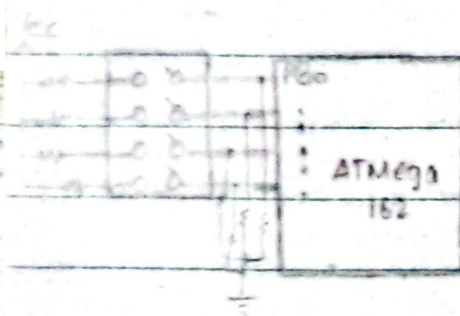
Register = ffr. n = pin number.

x = port name

Interfacing with DIP Switch.

DIP = Dual In Line Packages: Pins on dual sides of IC

→ Switch = Input device for micro-controller that is
 PB0 - PB7
 We will read the switch value at Port B.



○ = 0

(open)

(off)

● = 1

(close)

(on)

Jan 21, 2026

Diff. b/w:

push button: instantaneous 1 or 0 (can be configured)
 (pulse input)

Dip Switch, continuous 1 or 0.

Delay is important for push button.

LEDs: Forward Bias: LED Glows.

Anode: higher voltage

Cathode: lower voltage

Backward Bias: LED does not

glow

Anode: low voltage

Cathode: Higher Voltage

Anode



Cathode

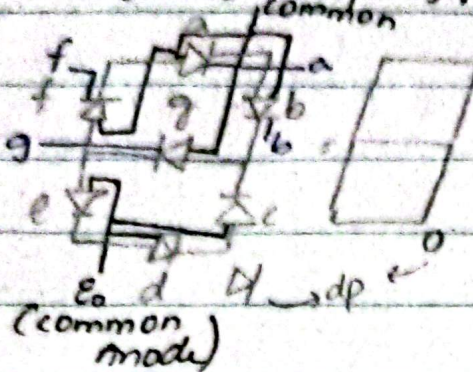
Diode
 (LEDs)

For LEDs, the Cathodes are connected to ground so
 as soon as anode gets any voltage, LED glows.
 otherwise LED must be faulty

7 Segment : 0-9 : decimal
0-9, A-F (16) hex
(0-14)

→ We can customize the 7-segment.

7. Segment Display:



- when showing a number, with the symbol of on LEDs as well.
- The upper 4 cathodes are given separate pins
- 0 must be provided to these pins to make it glow.
- Common pins are already provided Vcc.

1. int main(void) (We want to display 4)

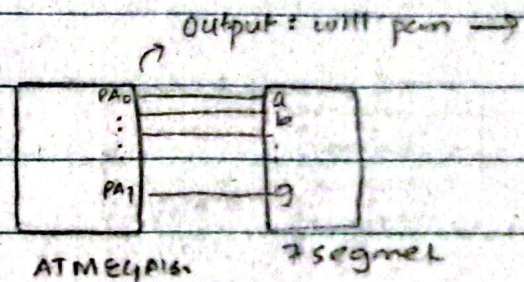
2. { DDRA = 0b111111; #1 indicates output port.

char num-to-be-displayed;

while(1) # dp g f e d c b a → low to glow

{ PORTA = 0b10011001; } →

#0 to on : common anode.



→ Display 7 then F in loop.

int main(void)

{ DDRA : 0b111111;

while(1)

dp g f e d c b a

-for BCD
ACBA

{ PORTA = 0b11111000; #7

-delay-ms(100) #ms milli second delay. -for BCD

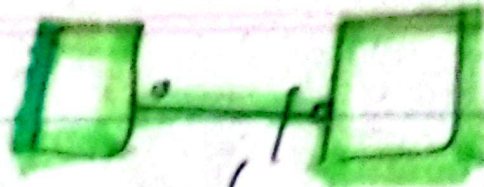
PORTA = 0b10001110; #F

-delay-ms(100)

}

Jan 23, 2026

DCBA
mLB 148



↳ We went there to be off

For BCD

POINT 7 6 5 4 3 2 1 0

 $x = 0 \text{ or } 1.$
$$x \quad 0 \quad 0 \quad 0 \quad 0 \quad 1 \quad 1 \quad 1 \quad \neq 7$$

x y z e o i o o #4

Everything remains same in the previous page code
 → Hex file from comp → USB → SPI Port → micro-controller interface
 serial prog. files

Jan 28, 2026.

[illegible]

Address Setup Time

Date: _____

① FP: Good Flat Package - LFC

→ Choosing a microcontroller

- ① meet the task + cost effective
- ② which bit will suit best
- ③ speed
- ④ Packaging: DIP or QUAD
- ⑤ Power consumption
- ⑥ Amount of RAM & ROM
- ⑦ Number of I/O pins
- ⑧ Ease of upgrade
- ⑨ Cost per Unit
- ⑩ Ease of development
- ⑪ Availability in present & future

→ RISC

→ AVR Controllers

→ ATMEGA 162

→ low power CMOS based on AVR.

→ Throughput 1 MIPS per MHz.

→ Read, Write, Modify

- Read Current Value of port of register
- modify specific bits
- Write it back

SBI: Set Bit in I/O register. Set two bits SBI and DDRA

CBI: Clear bit in I/O register. Clear two bits CBI and DDRA.

SBI register, bit

CBI register, bit

SBI DDRB, 2

→ set DDRB's bit 2 to 1.

PB2 becomes 1.

others remains unchanged.

→ drive values:

Output values on pins when configured as output.

→ pull-up resistors:

It is a weak resistor that makes sure that nothing float on a pin (0 or 1) when it is not connected to VCC or ground. Since, when unconnected the pin may read 0 or 1 randomly.

(Three Registers)

(R/W)PORTn: The values to be read or write Initial value: 0

(R/W)DDRA: Tells read (0) or write (1) Initial value: 0

(R)PINA: The actual voltage on PORTn (1 or 0) Initial value

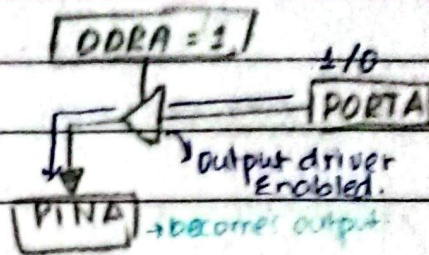
For port E: PORTD → PORTE2 (only) (3 pins) (35 I/O lines)

Date: _____

DDRA = 1: (Output)

Microcontroller drives the pin
→ Voltage on pin A is controlled by PORTA.

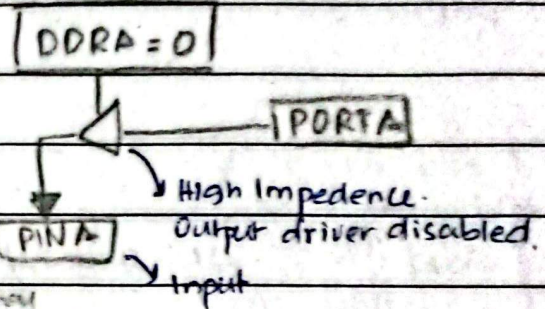
- PORTA = 1 → Pin drives 1 (High)
- PORTA = 0 → Pin drives 0 (Low)
- PORTA specifies pin A reads.



DDRA = 0: (Input)

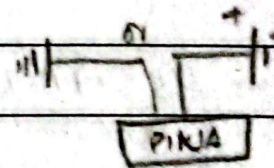
- Pull up gives default value of 1 if no external 1 or 0 given.
- Pull up disabled: read random 0 & 1.

- PORTA = 0: Pull up disabled external
- PORTA = 1: Internal pull-up enabled.
- PORTA = 0x100 switch for pull-up
- PINA reads external voltage on pin.



pin tied to ground: PINA = 0
" " " VCC: " 1

Floating + Pull up ON: 1
Floating + Pull up OFF: Undefines.



SFIO: Special Function I/O Register. (Control Register)

PUD: Bit 2 of SFIO → 1: All pull up disabled
→ 0: All pull ups enabled.

— Lecture 3, 4, 5 —

→ Red = 1 Blue = 0

→ DIP Switch 0 — 1

↑ VCC

#include <avr/io.h> → include avr header

#define F_CPU 1000000UL → clock freq. 1 MHz

#include <util/delay.h> → include delay library

external value given by dip switch

Date: _____

- Lecture 6 -

BCD to 7-segment Decoder IC:

→ $\overline{LT}$: Lamp test

Pin to test all LEDs at once, tests all segments at once.

$LT: 0$: All segment light up

$LT: 1$: normal operation (no test)

→ $\overline{BI}$: Turns off all segment at 0, 1 → normal operation

→ $\overline{RBI}$: Ripple blanking input: Low = suppress leading zeros.

$A_3 A_2 A_1 A_0$

DCBA

DDRA = 0b 1111 0100

$\overline{BI}$ $\overline{LT}$

ATmega	00	a
162	:	
	07	g

→ 0 to glow = common Anode.

From 74LS47

(lec 7.)

Date: _____

LCD display

16x2

= 16

= 16

each depicts one char = $16 \times 2 = 32$ characters

Each character is displayed (5x8) pixel dot

PE(0) = Enable pin to latch data to LCD

PE(1) = RW = Selects read/write mode for LCD

PE(2) = RS = Selects command or data for LCD

(lec 9) slide 384 → to do or not?

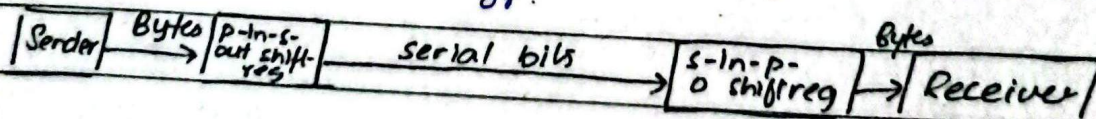
(LEC-19) **USART** Universal. Synchronous. Asynchronous.
Receiver. Transmitter.

It is used for serial communication between devices
such as:

Microcontroller ↔ Computer, Microcontroller ↔ Sensor,
Two Microcontrollers, Microcontroller ↔ GPS module.

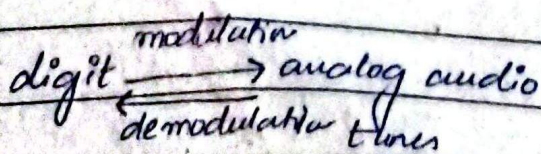
→ Serial method is used to transfer data many metres
away, this is not supported in parallel data transfer.

→ $\frac{\text{Byte}}{\text{of data}}$ → convert → serial bits 10100100 (8 bits) # Necessary for serial comm.
→ parallel-in-serial-out shift register



→ How to transfer the telephone call?

The bits are 0's & 1's, telephone needs audio of human
so we use modem (modulator-demodulator) for
this conversion.



Page No. _____

SYNCHRONOUS & ASYNCHRONOUS

- **Synchronous: (Clock Oriented)**
The sender & receiver use a common clock signal.
Data is sent continuously without start and stop bit.
→ Transmission is faster, structure is complex.
- **Asynchronous: (Byte Oriented) (Character Oriented)**
No common clock between sender & receiver.
Each data packet uses:
start bit, data bit, optional parity bit, stop bit.
→ Slightly slower, simpler.

USART is duplex

- **Simpler Communications:**
Data flow in only one direction. Transmitter → Receiver.
By radio broadcasting.

even parity = number of 1's in num
+ 1 parity must be even

→ **ASYNCHRONOUS.**

odd parity = odd

- **Framing:**
Process of dividing continuous stream of data between start & stop bit, smaller identifiable units called frames for transmission.

Frame = Actual Data + Control Info.

Framing is needed to let the sender & receive know where the data starts & ends.

start bit always 1 **stop bit** can be more than 1
(0 always) (1 always)

5: Data **Devices can use upto 8b of data D₀ - D₇**
1: Mark **Appt from start & stop bit.**

Baud Rate = Number of signal changes per second.

Why bps & baud rate diff?

Sometimes, a change in signal transfers several bits.
Baud = bits per second (in modem)

bps = baud rate (conductor wire)

DTE → DRE (Data Terminal Ready)

(DTE device) $\xrightarrow{\text{DTR}}$ (modem)

1: I cannot connect

I am ready to
communicate (mod)

MSB - MR - 3

16 → 11 USART Band set to be 10412 bps

UDR0A: Can't be status reg A
 UDR0B: USART Data Reg Empty
 ✓ UDR0 part of UDR0A
 1: Buffer busy
 0: Buffer empty
 here, both needed to send data

$$UBRR = \frac{F_{CPU}}{16 \times \text{Baud}} - 1 =$$

$$UBRR = 5$$

$$UBRR0H = 0x00$$

$$UBRR0L = 0x05$$

→ Data Set Ready: (DSR) (Active low) (upper bit) (low bit)

[Modem] → DSR → DTE device

- 1: I cannot enable conversion
- 0: I am ready for service

UCSR0B: enable TX/RX

UCSR0C: Frame Format

UBRR0(H/L) = Baud Rate

→ Request To Send: (RTS) (Active low)

[DTE device] → RTS → [Modem]

- 0: I have a byte to transfer
- 1: No byte to transfer

→ Clear To Send: (CTS) (Response to RTS)

[Modem] → CTS → [DTE device]

- 1: Okay! Send the data
- 0: Sorry! Cannot happen

→ Data Carrier Detect (DCD):

[Modem] → DCD → [DTE device]

- 1: Connection is active
- 0: No connection

$$UCS200 = 1$$

$$UCS201 =$$

$$UCSR0C = (1 < \text{USBS0}) | (3 < \text{UCS200})$$

2 stopbits: 1

1 stopbit: 0

Set stopbits

Review Question 7.

Total bps character: 1(stop bit) + 8b + 1(start bit) ⇒ 10b character

For 10,000 characters: 10 × 10,000 ⇒ 100,000 bits

× Overhead: 2 bits in 10 bits → $\frac{2}{10}$ bits in 1 bit

$$\frac{2}{10} \times 100,000 \text{ in } 100,000 \text{ bits}$$

Start bit always 1, not specific

We use 9600 bps

Time to transfer 1b → 960

In 1s → 9600 b

10,000 char → 2 × 10,000 bits of framing

$$\frac{20,000}{9600} = 2.08 \text{ s wasted of overhead}$$

$$\frac{1}{9600} \text{ s} \rightarrow 16 \text{ } \frac{100,000 \text{ s}}{9600} \rightarrow 10400 \text{ b}$$

100,000 b / will take 10.42s

$$\text{Overhead} = \frac{2}{10} \times 100 \Rightarrow 0.2 \%$$

UCS202	UCS201	UCS200	char bit
0	0	0	5
0	0	1	6
0	1	0	7
0	1	1	3
1	1	1	9

✓ DIP Switches

✓ LED

✓ Seven Segment without BCD

✓ Timer/Counter 0 (Normal mode with interrupts, CTC mode without interrupts) (lec 9-10) → lec 19-20

5. USART (interrupt based receive, timing diagrams, ^{calc} Baud rate)

6. I2C Protocol SCL Freq (lec 21-22)

7. Timing diagrams (interfacing, no embedded C code)

Keypad?? (Lec 11-12)

XLEC 13-14

XLEC 23-24

DAC

+ BCD

NO theory + MOTORC + ADC + SPI + LCD + external

8. On chip EEPROM based application

no embedded
→ code

21 (code) External memory Interface (like in class example)

9. dc optocoupler based apps (lab 14)

10. AVR GPIO reg. like DDRX PORTX PINX

must be uno

→ included or not?

✓ Keypad (Lec 11-12), LEC 15, (Keypad + LCD)

For I2C: Interface multiple I2band device with microcontroller.

$$\text{Baud Rate} = \frac{f_{osc}}{16(UBRR+1)} \quad \text{Asynchronous Normal Mode}$$

$$" \quad " \quad = \frac{f_{osc}}{8(UBRR+1)} \quad \text{Asynchronous double speed}$$

$$" \quad " \quad = \frac{f_{osc}}{2(UBRR+1)} \quad \text{Synchronous Master Mode}$$

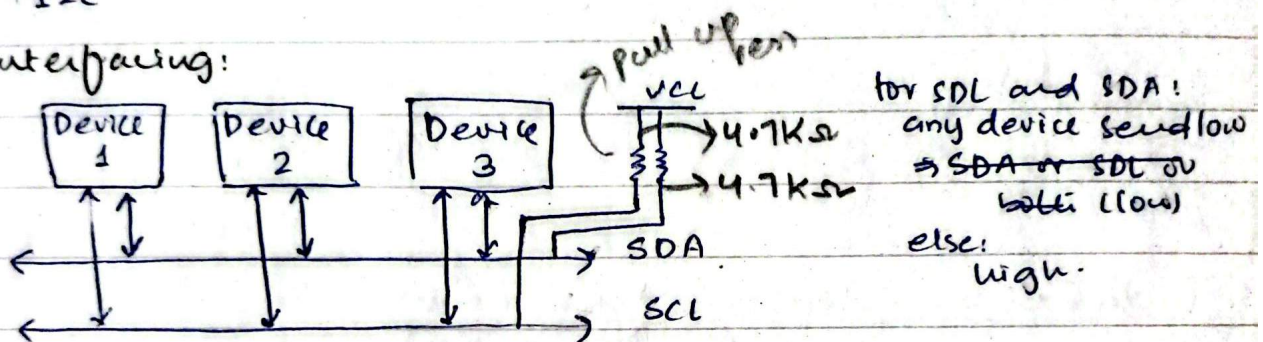
→ 6 ✓

→ For understanding
(No mark)

LEC 21-22 I2C PROTOCOL (No need of Handout)

- I2C (Integrated Inter circuit) or I^2C or IIC or TWI serial communication protocol, for communication b/w electronic devices.
- Uses two wires. Serial Data Line (SDA), Serial Clock Line (SCL)
- Uses master-slave architecture, supports multiple devices on the same bus.
- Ideal for short-distance comm.
- Connection-oriented-communication with acknowledge I2C

→ Interfacing:



- Devices can send 0 (Low) only on data lines. Because, if one sends high & other low: conflict. Hence, only low can be sent.

→ How to make lines high:

When no device sending, low, pull-up resistor makes the line high.

→ This behavior is wired and:

$$0 \cdot 0 = 0 \quad 1 \cdot 1 = 1$$

- 120 devices can share the Bus (SDA, SCL)
- Each device is node. Node → Master

- Master: Generates clock signal, initialises / terminates the system
- Slave: Receives the clock signal.
- Both master & slave can receive & transmit data.

Four Modes of operation:

- Master Transmitter
- Master Receiver
- Slave Transmitter
- Slave Receiver.

→ BIT FORMAT:

SDA: Carrier bit SCL: Carrier clock pulses

SCL = High: Read SDA bit

SCL = Low: Change SDA bit

→ Each SDA High → read SDA bit (1) → SCL low → SDA changes its bit to the next bit → SCL High again to read SDA bit.

→ START & STOP CONDITIONS:

Normally SDA changes, when SCL low, but when SCL High,

SDA: 1 → 0: Start indication

SDA: 0 → 1: Stop indication

start — busy — stop

Repeated Start:

Master starts another communication without ending the previous one.

→ PACKET FORMAT IN I2C:

On the 9th bit:

SDA (transmitter) releases: — (not controlling the bus)

SDA (receiver): → 0:  : Acknowledge

→ 1:  : NACK, No Acknowledge

SDA Aggregates: combined effect of receiver & transmitter.

SDA (transmitter): what transmitter is doing with SDA

SCL (Master): Master generates the clock pulses

transmitter sending & changing bit on each pulse except the null.

→ ADDRESS PACKET FORMAT:

RW/W: 8th bit → 1: Master reads the next data from the slave address.
→ 0: Master will write to this slave the next data frame.

Address Packet: Bits: 0 — 6 (Address) + 7 (R/W) + 8 (Ack)

Data Packet: Bits: 0 — 7 (Data) + 8 (Ack)

→ DATA PACKET FORMAT:

→ No more data to be received, last byte received.

SDA line High: NACK.

→ Clock Stretching:

NACK: Last Data Byte received.

Stretching: No need of last data byte too to delay the data & not miss it.

→ Arbitration:

Two masters initial at the same time.

→ level of bus is compared with what it expects (each transmitter) the one matches becomes master the rest slaves.

Example 18.5: Pg 636

SCL 1 2 3

A O O, 1,

B O O O

SDA
(~~++~~) and: 0 0 0

Master A after loosing, will not control SDA & will carry exact bit as B.

↳ equivalent to B : B = master, A = slave.

→ multiByte Burst Read:

write to the location of slave addr. the start addr., then read.

LEC 27 - EEPROM (Embedded C Code)

→ EEPROM Register.

stores address of EPPROM location you want to access

1 << EEWE : 00001 : 00100 & EEER $\Rightarrow$ $\begin{matrix} \rightarrow \text{is enable} \\ \rightarrow \text{UI decoder} \end{matrix}$
tells actual value of EEWE bit in EEER

LSC 25-26

→ External Memory Interface

Hardware feature of micro-controller that allows it to connect & access external memory. (Like RAM, ROM, flash)

→ It lets micro-controller read from external RAM
write to external RAM.

Address Bus: selects memory location. (~~A0-A3~~)

Dalā bus: Carrier dalā (50-07)

RD:

Rw:

ALZ:

ca:

→ When XMEM enabled, micro-controller can use external RAM.

→ AD7:0

These 8 plus carry:

lower 8 addr. bits (A0 - A7) and lower data bits (D0 - D7)
not necessarily.

→ A15:8: Upper 8 bits of Address

→ ALE (Address Latch Enable)

0: AD7:0 → contains address data

1: AD7:0 → contains address

→ Read Strobe $\overline{RD}$: CPU to read from external memory.

→ Write Strobe $\overline{WD}$: CPU write to external memory.

→ 74x573 Latch.

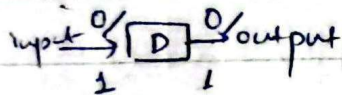
Input: D0-D7 Output: Q0-Q7 Enable pin = E

when E = 1: latch is transparent

E = 0: Latch freezes

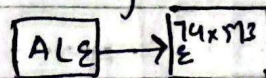
Last output will remain as it

is no matter how the input changes



input = 1101

output = 1101



Hence, we can freeze the data lines - like → AD0-AD7

ALE = 1 → CPU places lower address on 00110100 → Latch output

00110100 → ALE = 0, Latch freeze → Now AD0-AD7 11111111 →

Latch output still 00110100

→ Memory size determines how many address lines needed.

λ: SRAM1 → 8KB

n addr line = 2^n memory locations

SRAM2 → 4KB

4KB $\Rightarrow 4 \times 1024 \Rightarrow 4096 \Rightarrow 2^{12} \Rightarrow 12$ addr line (A0-A11)

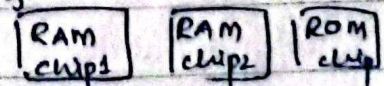
8KB $\Rightarrow 8 \times 1024 \Rightarrow 8192 \Rightarrow 2^{13} \Rightarrow 13$ addr lines (A0-A12)

Remaining

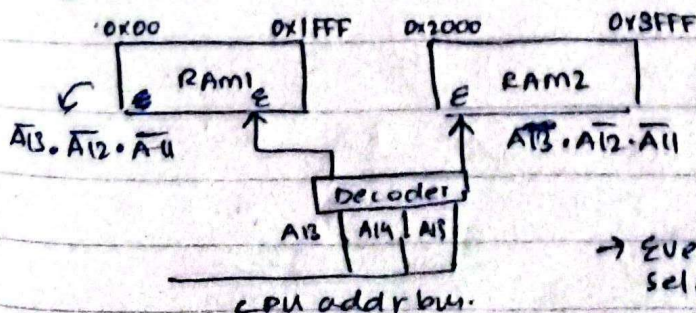
→ Chip Select:

Hence, only one chip must be enabled at a time

eg:



all these share data & addr. bus



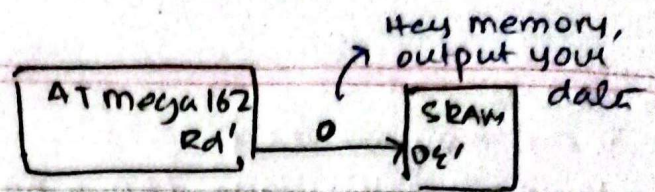
→ Upper bits A15 — A12
→ A15, A14, A13 → decision
0 0 0 RAM1
0 0 1 RAM2

→ Every memory has its own chip select signal with decoding logic

→ output Enable (OE')
 part of SRAM.

→ connected to RD' of ATmega162

→ Allows memory chip to put data on data bus to read.



→ WRITE ENABLE (WE')
 part of SRAM.

→ connected WR'

→ tells memory chip to store incoming data.

